A Computer Vision Project Report

MIDTERM PROJECT REPORT

Submitted as a requirement for the course
Computer Vision

Group Project Specialization

Object Sorting

Prepared by:

Villamer, Gracielle Ann Z.
Pimentel, Deiven Joseph I.
Romance, Jeric E.
Ibarrientos, Jenny J.
Namia, Erica Denese L.
Biares, Francisc Louise Q.

BS Computer Science – 4B

Submitted to:

Elena Clarexe Gulane
Course Instructor

1. INTRODUCTION

1.1 Project Overview

The system recognizes four object classes — bottle, can, box and ball — from a live camera feed, draws a box around the object and labels it with its class and confidence. It is the vision component of a Visual Object Sorting Robot: in the Final project the recognized class will decide which bin the robot moves the object to.

The pipeline has two stages. A YOLO-World detector (yolov8s-worldv2) proposes where objects are in the frame, and a MobileNetV2 classifier trained by the group decides what each object is. A lightweight tracker then smooths the result across frames so the label stays steady.

1.2 Problem Description

A sorting robot must know what an object is before it can decide where it goes. Doing this from a camera is hard because the same class looks very different from one example to the next: a ball can be a golf ball or a basketball, and a bottle can be full, empty or crushed. The camera also sees hands, the table and background clutter, which the system must not mistake for objects.

The Computer Vision problem is therefore object localization + image classification under changing lighting, angle, scale and background, fast enough to run on a laptop CPU in real time.

1.3 Project Objectives

General objective: Develop a camera-based system that detects and correctly classifies objects into four sorting categories in real time.

Specific objectives:

1. Build a labelled dataset of at least four object classes (bottle, can, box, ball), split into training and validation sets.
2. Train a MobileNetV2 transfer-learning classifier that reaches at least 90% validation accuracy across the four classes.
3. Localize objects in a live 640×480 webcam feed with a YOLO-World detector and classify each candidate box.
4. Display the result as a coloured bounding box, class label and confidence score, marked CONFIRMED after 5 consistent detections.
5. Report accuracy, precision, recall, F1-score and a confusion matrix.

1.4 Scope and Limitations

Item	**Covered in the Midterm**
Classes / categories	Bottle, can, box, ball (4 classes)
Objects / targets	Plastic bottles (incl. crushed), beverage cans, cardboard/product boxes, 9 ball types (baseball, basketball, billiard, bowling, cricket, football, golf, tennis, volleyball)
Visual tasks	Object localization (bounding box), classification, confidence scoring, frame-to-frame tracking
Operating scope	One object at a time in front of a webcam, indoor lighting, 640×480 input, laptop CPU
Input modes	Live webcam or a single still image

Limitations:

- The robot (chassis, motors, arm or pusher) is not yet built; the Midterm covers only the vision component.
- The system reports one object per frame — the most confident one — not several objects at once.
- The trained model has no other class, so a hand or unrelated item may be labelled as one of the four classes if its confidence passes 80%.
- Most training images come from public datasets, not from the group's own camera setup.
- Objects outside the four classes are out of scope.

1.5 Group Project Specialization

The group is assigned **Visual Object Sorting Robot**. The robot must identify objects by their visual characteristics and place them in the correct category. The primary CV problem is **object classification**; the Midterm requirement is to **recognize 4 object classes**, and the Final requires the robot to identify the object, decide its destination and move or sort it.

2. SYSTEM DESIGN AND ARCHITECTURE

2.1 Overall System Architecture

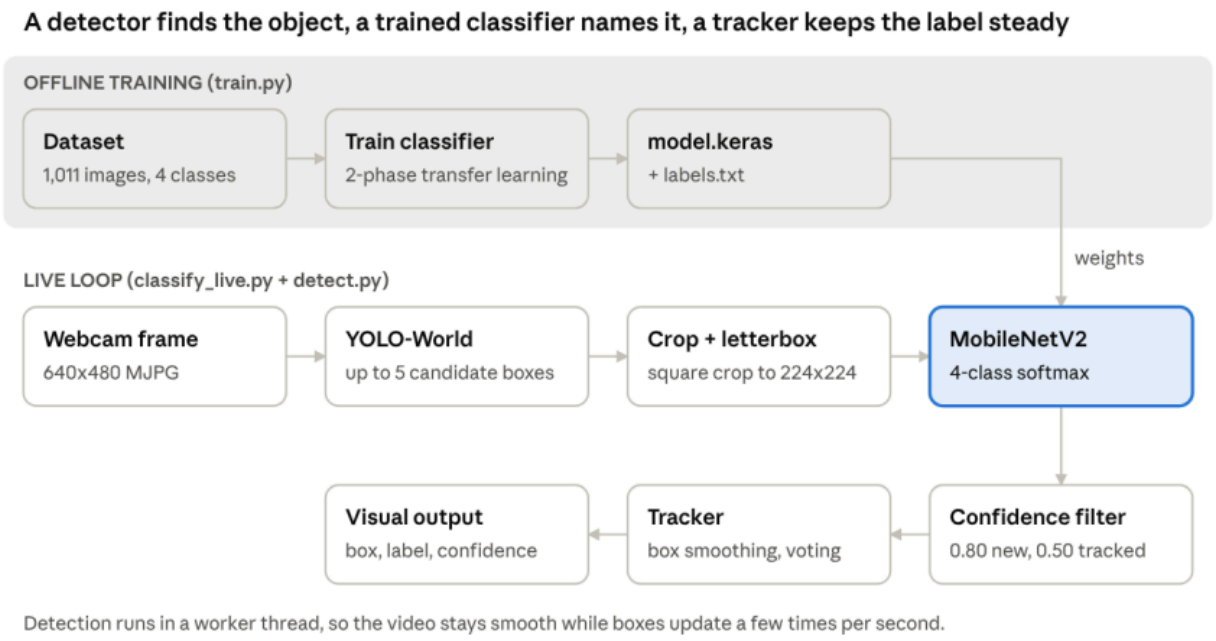


Figure 1. System architecture · offline training and live recognition loop

YOLO-World only proposes *where* objects might be (with a low 0.10 threshold); the group's MobileNetV2 classifier makes the final decision on *what* each one is. Splitting the job this way lets a general detector handle cluttered scenes while a small, task-specific model handles the four sorting classes.

2.2 Computer Vision Workflow

1. **Capture** — the webcam is opened at 640×480 in MJPG format with a 1-frame buffer, so the newest frame is always used.
2. **Propose** — a worker thread passes the newest frame to YOLO-World (yolov8s-worldv2.pt, prompts: bottle, can, box, ball, input size 416). Up to 5 boxes above 0.10 confidence are kept; slivers under 4 px and boxes over 90% of the frame are dropped.
3. **Crop** — each box is enlarged by 15% padding into a square crop, then letterboxed (padded, not stretched) to 224×224 and converted BGR→RGB.
4. **Classify** — all crops go to MobileNetV2 in one batch; the softmax gives a probability for each of the 4 classes.

5. **Select** — a new object needs ≥ 0.80 confidence; a box overlapping the tracked object (IoU > 0.3) needs only ≥ 0.50 and gets a +0.15 score bonus. The highest-scoring box wins.
6. **Track** — the box is smoothed over time, the label is a confidence-weighted vote over the last 7 checks, and the object is held for 0.8 s after it is last seen.
7. **Display** — a class-coloured box, the label and confidence are drawn; after 5 agreeing checks the box thickens and shows CONFIRMED.

2.3 Hardware Components

Component	Specification / Model	Purpose
Camera	Built-in HD laptop webcam, used at 640×480 MJPG	Captures live frames of the object
Controller	Laptop, CPU inference (Intel **Core vPro i5**, 16 GB RAM)	Runs YOLO-World, MobileNetV2 and the display
Robot chassis, motor driver, motors, power source	Not yet used	Planned for the Final (sorting mechanism)

2.4 Software and Development Tools

Software / Tool	Version, if applicable	Purpose
Python	3.9–3.12	Main programming language
OpenCV	≥ 4.8	Camera capture, cropping, GrabCut, drawing results
TensorFlow / Keras	TF ≥ 2.13 (model saved with Keras 3.15.1)	MobileNetV2 training and inference

Ultralytics (YOLO-World)	≥ 8.3, weights `yolov8s-worldv2.pt`	Open-vocabulary object proposals
Numpy	≥ 1.24	Array maths, IoU, confusion matrix
Pillow	≥ 10.0	Image file support
Arduino IDE	Not used yet	Planned for robot control in the Final

2.5 Final Robot Decision and Control Workflow

Final Project only.

Explain how Computer Vision results are converted into robot decisions and actions.

The workflow should demonstrate:

SEE → RECOGNIZE / INTERPRET → DECIDE → ACT

A flowchart or system diagram is strongly recommended.

3. DATASET

3.1 Dataset Description

The dataset holds **1,011 RGB images in 4 classes**, stored one folder per class (`dataset/ball`, `dataset/bottle`, `dataset/box`, `dataset/can`) and shipped as `dataset/dataset.rar` (about 35 MB). Images come in mixed sizes and aspect ratios; the training script letterboxes them to 224×224 instead of stretching them.

3.2 Target Classes / Categories

Class / Category	Description	Number of Samples

Bottle	Plastic bottles of different shapes, including 100 crushed bottles	300
Ball	9 ball types, 33 images each: baseball, basketball, billiard, bowling, cricket, football, golf, tennis, volleyball	297
Box	Cardboard and product boxes, several angles and rotations	260
Can	Beverage cans	154
Total		**1,011**

3.3 Dataset Collection and Source

The images were gathered from public image datasets and organized by the group into the four class folders. File names show the origins: the box images are Roboflow exports (*_jpg.rf.<hash>.jpg, with rotated augmented copies), bottles come from a “Bidon” bottle set plus a crushed-bottle set, cans from a beverage-can set, and balls from a sports-ball set.

The project also includes collect_data.py, which saves crops from the group's own webcam: the detector outlines the object and keys **1–5** save it as bottle, can, box, ball or other. This tool supports adding images from the real sorting setup.

3.4 Dataset Distribution

Bottle (29.7%), ball (29.4%) and box (25.7%) are close in size, while can is the smallest class at 15.2% — about half the size of bottle. To keep the model from favouring the larger classes, train.py applies class weights computed as total ÷ (classes × class count):

Class	**Training images**	**Class weight**
Ball	238	0.85
Bottle	240	0.84

Box	208	0.97
Can	124	1.63

3.5 Training, Validation, and Testing Sets

train.py performs a **stratified 80/20 split** per class with a fixed random seed (42), so every class keeps the same proportion in both sets.

Dataset	**Number of Samples**	**Percentage**
Training	810	80.1%
Validation	201	19.9%
Testing	0 (live camera and still-image tests only)	—
Total	**1,011**	**100%**

- **Training set** — used to update the classifier's weights.
- **Validation set** — used after every epoch for early stopping and learning-rate reduction, and for the final accuracy, precision, recall, F1-score and confusion matrix (Section 6.3).
- **Testing set** — no separate offline test set is held out yet; the system was instead tested live with the webcam (Sections 6.4 and 6.5). Because the validation set also steers early stopping, its accuracy may be slightly optimistic. A separate test set is planned for the Final (Section 9.2).

3.6 Dataset Preparation and Preprocessing

Only the steps actually performed by train.py are listed:

1. **File filtering** — only .jpg, .jpeg, .png, .bmp and .gif files are loaded; .webp/.tif files are counted and skipped.

2. **Minimum-size check** — training stops if any class has fewer than 5 images.

3. **Decoding** — every image is decoded as 3-channel RGB; corrupt files are dropped automatically (ignore_errors).

4. **Letterboxing** — resize_with_pad scales each image to 224×224 while keeping its aspect ratio, filling the rest with black.

5. **Shuffling and batching** — the training set is reshuffled every epoch; batches of 16 are prefetched.

6. **Augmentation (training only, inside the model)** — random horizontal flip, rotation up to ±36°, zoom ±25%, shifts ±15%, brightness ±30%, contrast ±30%, then Gaussian noise (σ = 0.05).

7. **Normalization** — MobileNetV2's `preprocess_input` scales pixels from 0–255 to −1…1.

4. COMPUTER VISION METHODOLOGY

4.1 Computer Vision Approach

The group uses a **two-stage detect-then-classify approach**: an open-vocabulary detector (YOLO-World) localizes candidate objects, and a transfer-learned CNN (MobileNetV2) classifies each candidate into bottle, can, box or ball.

This suits object sorting for three reasons:

- **Classification is the core task.** Specialization 8 is defined as object classification; a dedicated classifier trained on the group's own classes is more reliable for these four categories than a general detector's labels alone.
- **Localization is needed for the robot.** The box tells the Final robot where the object is, and cropping to it removes most of the background before classification.
- **It is light enough for a laptop CPU.** MobileNetV2 is built for mobile devices, and transfer learning from ImageNet gives good accuracy from only about 1,000 images.

4.2 Image Processing / Preprocessing

Technique	**Where**	**Parameters**
Camera setup	`classify_live.py`	640×480, MJPG codec, buffer size 1 (DirectShow on Windows)
Square crop with padding	`detect.crop_square`	15% padding on each side, minimum 32 px, kept inside the frame
Letterbox resize	`letterbox` / `resize_with_pad`	224×224, aspect ratio kept, black padding
Colour conversion	`classify_batch`	BGR (OpenCV) → RGB (model)
Normalization	inside the model	`mobilenet_v2.preprocess_input`, pixels to [−1, 1]

GrabCut outline (optional)	detect.refine_outline	ROI padded 10%, max side 220 px, 3 iterations; rejected if mask < 15% of the box
Background subtraction (fallback mode)	detect.Localizer	Grayscale + 21×21 Gaussian blur, difference threshold 30, 5×5 morphological close/open, blobs ≥ 1% of the frame

4.3 Model / Algorithm

The system combines a pretrained detector with a trained CNN classifier.

Detector — YOLO-World (yolov8s-worldv2). An open-vocabulary YOLOv8-S model that finds objects described by text prompts. The prompts are set to bottle, can, box, ball. It is used as-is (not retrained) and only proposes boxes; its labels are not used.

Classifier — MobileNetV2 transfer learning. The trained model (model.keras) has these layers:

1. Input 224×224×3
2. Augmentation block (active only in training)
3. MobileNetV2 preprocessing (scale to [−1, 1])
4. Gaussian noise, σ = 0.05 (training only)
5. MobileNetV2 backbone, ImageNet weights, no top layer
6. Global average pooling → 1,280 features
7. Dropout 0.3
8. Dense layer, 4 units, softmax → ball, bottle, box, can

Fallback localizers. If YOLO-World cannot load, --mode auto switches to background subtraction against a saved empty-scene photo; --mode none classifies the whole frame.

4.4 Model Training / Development

Training runs in two phases with python train.py:

Setting	Phase 1 — train the head	Phase 2 — fine-tune

Trainable layers	New classifier head only (backbone frozen)	Head + last 30 backbone layers
Max epochs	12	10
Optimizer	Adam	Adam
Learning rate	1×10^{-3}	1×10^{-5}
Batch size	16	16
Loss function	Sparse categorical cross-entropy	Sparse categorical cross-entropy
Class weights	Yes (Section 3.4)	Yes
Early stopping	val accuracy, patience 3, best weights restored	same
LR schedule	ReduceLROnPlateau on val loss, ×0.5 after 2 epochs	same

Random seeds for Python, NumPy and TensorFlow are fixed at 42 so the split and training are repeatable. The ImageNet weights are downloaded on the first run. The saved model was produced on 19 October 2026 (Keras 3.15.1).

4.5 Model Parameters and Configuration

Parameter	Value	Effect
`YOLO_CONF` (`--det-conf`)	0.10	Low on purpose: detector only proposes; the classifier decides
`YOLO_IMGSZ` (`--imgsz`)	416	Detector input size; 320 is faster
Max candidates	5	Boxes classified per check
`CONF_THRESHOLD` (`--conf`)	0.80	Needed to start tracking a new object

KEEP_THRESHOLD (--keep-conf)	0.50	Needed to keep an object already tracked
On-track bonus	+0.15 when IoU > 0.3	Favours the object already followed
New-object reset	IoU < 0.2	Starts over when a different object appears
HOLD_SECONDS (--hold)	0.8 s (or 3× the detection interval)	Keeps the box through brief misses
VOTE_WINDOW	7 checks	Label = confidence-weighted vote; changes only if a rival is 1.3× stronger
STABLE_FRAMES	5 checks	All agree → CONFIRMED
Box smoothing	0.6 per detection, 0.4 per video frame	Steadies the drawn box

4.6 Detection / Recognition Process

Camera Frame → YOLO-World Candidate Boxes → Square Crop + Letterbox (224×224) → MobileNetV2 Prediction → Confidence Filter → Tracker Vote → Class Label + Bounding Box

For a still image (`python classify_live.py photo.jpg`), the same steps run once without tracking, and the result is saved as `photo_labeled.jpg`.

5. SYSTEM IMPLEMENTATION

5.1 Computer Vision Implementation

The system is written in Python and split into four scripts, each with one job:

File	**Role**

collect_data.py	Opens the camera, outlines the object with the detector, and saves a clean square crop to dataset/<class>/ when keys 1–5 are pressed
train.py	Loads the dataset, splits it 80/20, trains MobileNetV2 in two phases, prints accuracy and a confusion matrix, saves model.keras and labels.txt
detect.py	The Localizer class (YOLO-World, background subtraction or whole-frame), square cropping, GrabCut outline and drawing of results
classify_live.py	Runs recognition on a webcam or a single image, with the confidence filter, Tracker and Worker thread

Key implementation decisions:

• **Detector and classifier are separate.** YOLO-World runs with a low threshold so it rarely misses an object; the classifier, trained on the group's own classes, filters out weak proposals.

• **All candidates are classified in one batch.** Up to 5 crops go through MobileNetV2 in a single call, which is faster than one call per crop.

• **Video and detection run on separate threads.** The main loop shows every camera frame, while a Worker thread always takes the newest frame for detection. Old frames are skipped, so the display never lags behind.

• **A warm-up inference** runs at start-up so the first real frame is not slow.

• **The tracker favours stability.** Smoothing, holding and voting stop the box and label from flickering when one check fails.

5.2 Input and Camera Processing

Frames come from a webcam opened through OpenCV (--camera selects the device; DirectShow is used on Windows for faster start-up). The camera is set to 640×480 MJPG with a one-frame buffer. Each frame is shown immediately and also submitted to the detection thread, which replaces any frame still waiting.

A single image can be processed instead by passing its path: python classify_live.py photo.jpg.

5.3 Visual Output

The result is drawn on the live video window classify_live:

- **Bounding box** in a class colour: bottle = blue, can = yellow, box = green, ball = red, unknown = grey.
- **Label and confidence** on a filled tag above the box, e.g. bottle 94%.
- **CONFIRMED** added to the tag, with a thicker (3 px) box, after 5 agreeing checks.
- **“No object”** when nothing passes the confidence filter.
- **Speed readout** at the bottom: camera FPS and detection checks per second (e.g. cam 30 fps | det 5.2/s).

All four classes were recognized and confirmed in live webcam tests, each at 97–99% confidence, with the camera at 15 fps and detection at 4.2–6.7 checks per second.

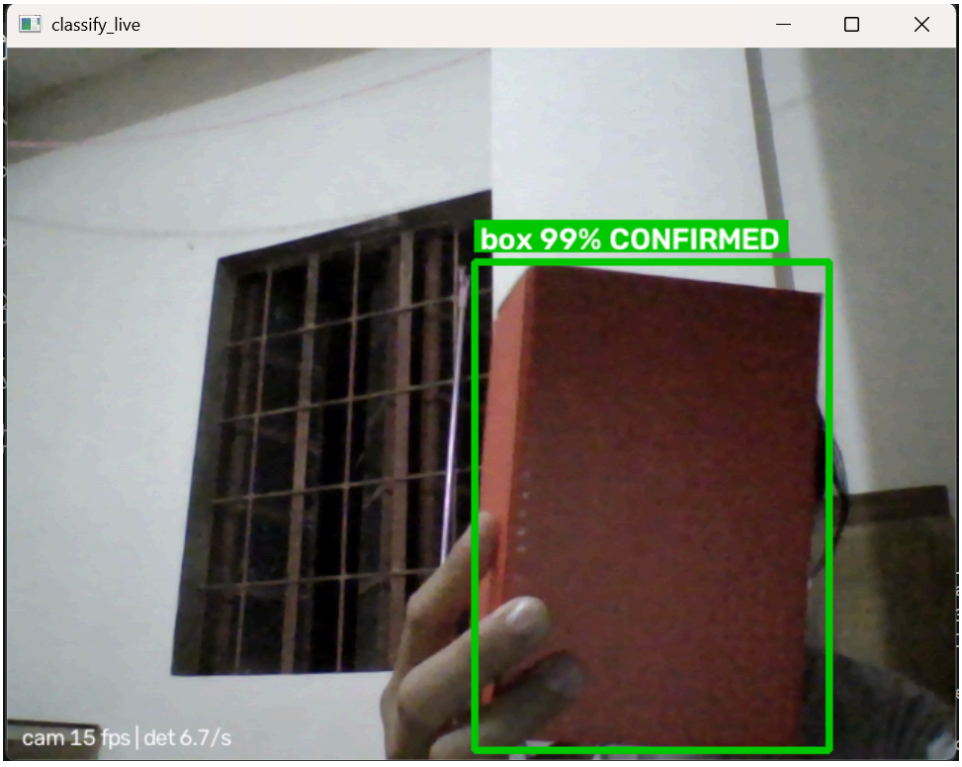


Figure 2. A red product box held in front of the webcam is labelled **box 99% CONFIRMED** with a green bounding box (det 6.7/s).

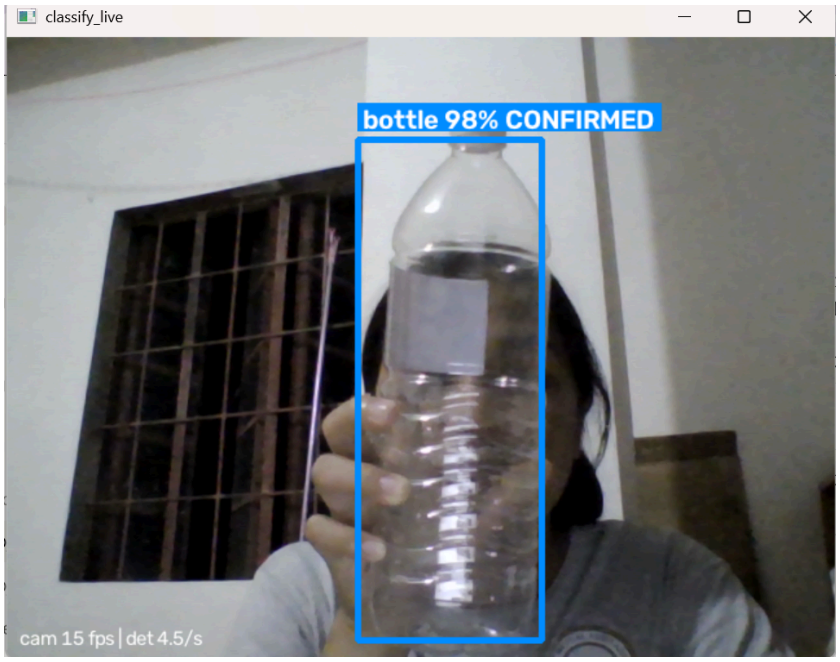


Figure 3. A clear plastic water bottle is labelled **bottle 98% CONFIRMED** with a blue bounding box (det 4.5/s).



Figure 4. A canned-tuna can is labelled can 97% **CONFIRMED** with a yellow bounding box (det 4.2/s).

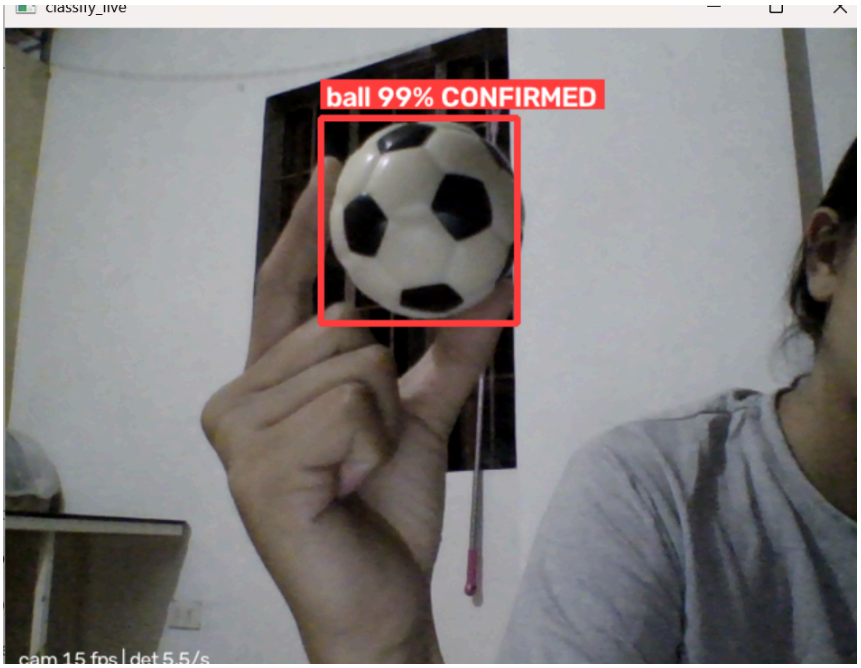


Figure 5. A small soccer-style ball is labelled ball 99% **CONFIRMED** with a red bounding box (det 5.5/s).

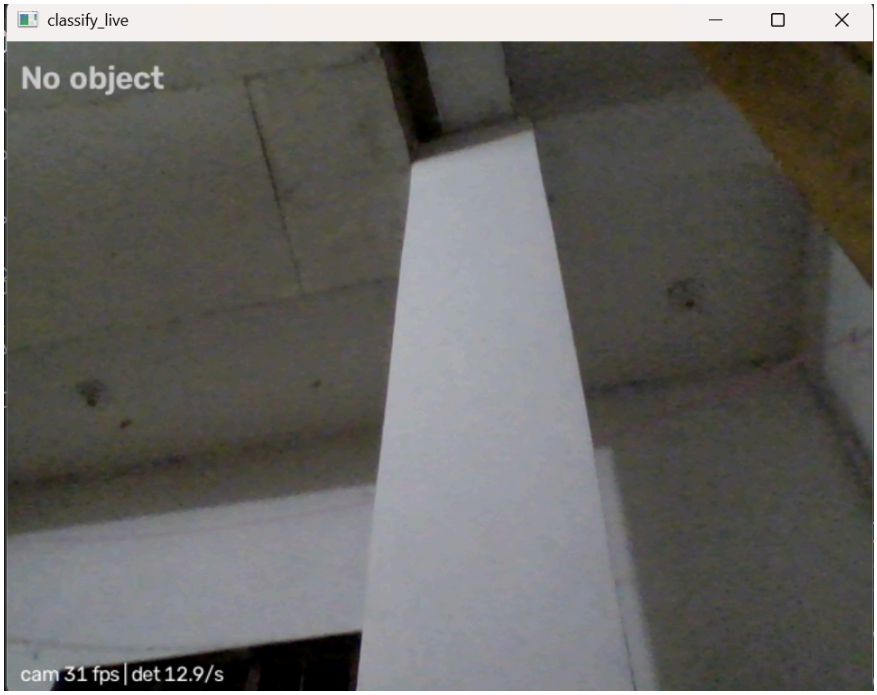


Figure 6. With only walls and a wooden beam in view, the system shows **No object** and draws no box. Detection runs faster here (det 12.9/s, cam 31 fps) because no candidate boxes need classifying.

5.4 Decision-Making and Control Logic

Final Project only.

Explain how the system converts Computer Vision results into decisions. A decision table, pseudocode, or flowchart may be included.

5.5 Robot Integration

Final Project only.

Explain how the Computer Vision system communicates with and controls the physical robot.

Describe the integration of the:

> Camera → Computing System → Decision Logic → Controller → Motor Driver → Motors

6. TESTING AND EVALUATION

The offline results in Section 6.3 were produced by `evaluate.py`, which rebuilds the same 80/20 validation split as `train.py` (seed 42) and runs the trained `model.keras` on the 201 validation images. The live results in Section 6.4 come from `classify_live.py` on the group's webcam.

6.1 Computer Vision Testing

The system was tested in two ways:

1. **Offline validation** — `evaluate.py` runs the trained model on the 201 held-out validation images and reports overall accuracy, precision, recall, F1-score per class and a confusion matrix.
2. **Live demonstration tests** —each class was shown to the webcam with `python classify_live.py` under normal indoor lighting. For each trial the group recorded the displayed label, the confidence, and whether CONFIRMED appeared. Empty scenes, objects outside the four classes, and a very dark room were also tested (Section 6.5).

6.2 Evaluation Metrics

- **Accuracy** — correct predictions ÷ all predictions.
- **Precision** per class — of the images predicted as a class, how many truly belong to it.
- **Recall** per class — of the images of a class, how many were found (reported by evaluate.py)

- **F1-score** — the balance of precision and recall: 2 × P × R ÷ (P + R).
- **Confusion matrix** — which classes are mistaken for which.
- **Live recognition rate** — correct CONFIRMED labels ÷ live trials.
- **Speed** — detection checks per second (`det` readout).

6.3 Training, Validation, and Testing Results

The classifier reached **95.5% validation accuracy (192 of 201 images correct)**, with an average F1-score of 96.0%.

Confusion matrix (rows = true class, columns = predicted class):

True \ Predicted	Ball	Bottle	Box	Can	Recall
Ball (59)	**58**	1	0	0	98.3%
Bottle (60)	5	**54**	0	1	90.0%
Box (52)	2	0	**50**	0	96.2%
Can (30)	0	0	0	**30**	100.0%

Per-class metrics:

Class	Precision	Recall	F1-score
Ball	89.2%	98.3%	93.5%
Bottle	98.2%	90.0%	93.9%
Box	100.0%	96.2%	98.0%

Can	96.8%	100.0%	98.4%
Average	**96.0%**	**96.1%**	**96.0%**

Can and box are recognized almost perfectly (F1 98.4% and 98.0%). Most errors fall into the **ball** column: 5 bottles and 2 boxes were predicted as ball, which lowers ball's precision to 89.2% and bottle's recall to 90.0%.

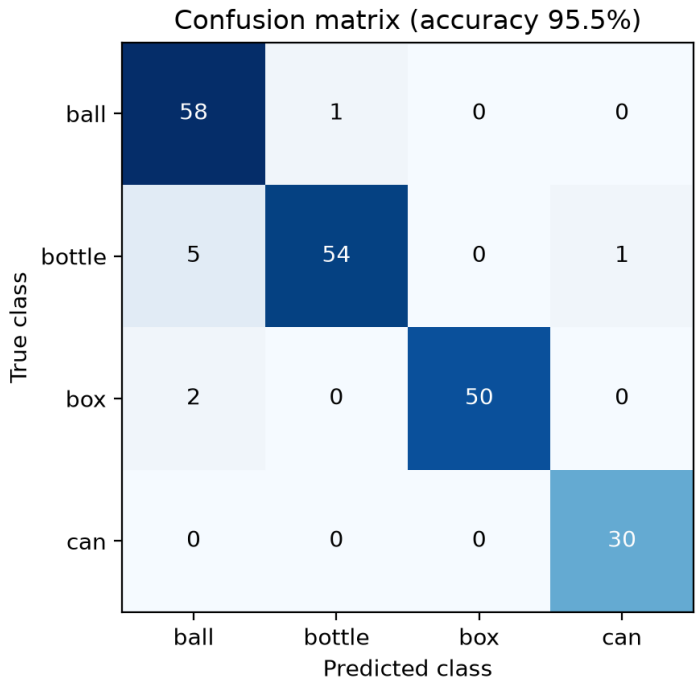


Figure 7. Confusion matrix on the 201 validation images (accuracy 95.5%). The dark diagonal shows correct predictions; most errors fall in the ball column.

6.4 Sample Detection / Recognition Results

Test	Object shown	Displayed result	Confidence	Confirmed?	Correct?	Notes
1	Red product box, upright, held in hand	box	99%	Yes	Yes	Figure 2; hand not included in the box

2	Clear plastic water bottle, upright	bottle	98%	Yes	Yes	Figure 3; transparent object still recognized
3	Canned tuna, label facing camera	can	97%	Yes	Yes	Figure 4; lowest confidence of the four
4	Small soccer-style ball	ball	99%	Yes	Yes	Figure 5
5	Empty scene (walls, no object)	No object	—	—	Yes	Figure 6; no false box drawn
6	Open hand only, no object	No object	—	—	Yes	Figure 11

All four classes were recognized correctly, and the empty scene produced no false box. An unsuccessful case is shown in Section 6.5 (Figure 10): an object held in a very dark room was not detected.

6.5 Unseen Situations

The system was tested with objects and conditions that are not in the dataset. Unknown objects were rejected as “No object” instead of being forced into one of the four classes; only very dark lighting caused a failure.

Test	Situation	Displayed result	Correct?	Notes

U1	Toothbrush held in hand (not a trained class)	No object	Yes	Figure 8; neither the toothbrush nor the hand was labelled
U2	Pink kitchen strainer, round like a ball (not a trained class)	No object	Yes	Figure 9; the round shape did not trigger a false “ball”
U3	Object held in a very dark room	No object	No	Figure 10; frame too dark for the detector to find the object
U4	Open hand only (not a trained class)	No object	Yes	Figure 11

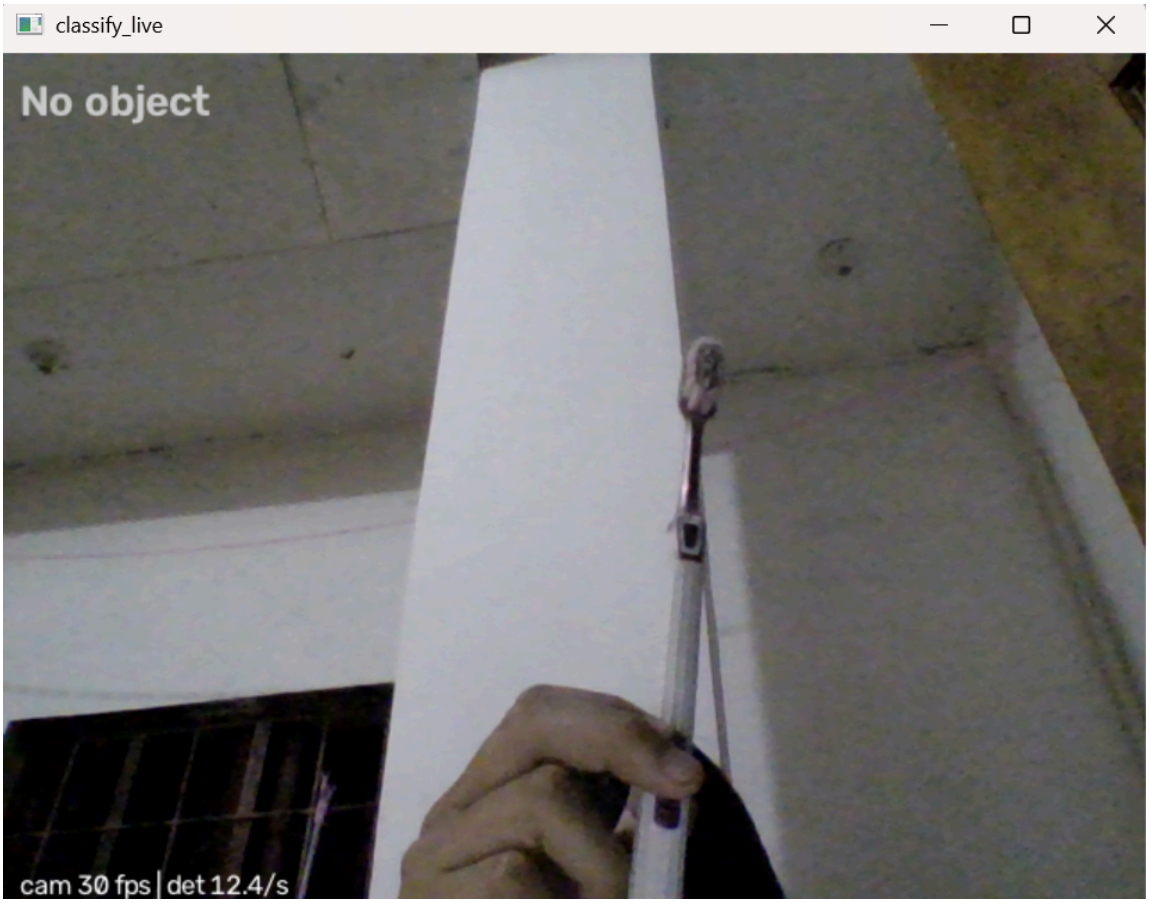


Figure 8. A toothbrush, which is not one of the four classes, is held in front of the camera. The system shows **No object**: both the toothbrush and the hand holding it are correctly ignored.

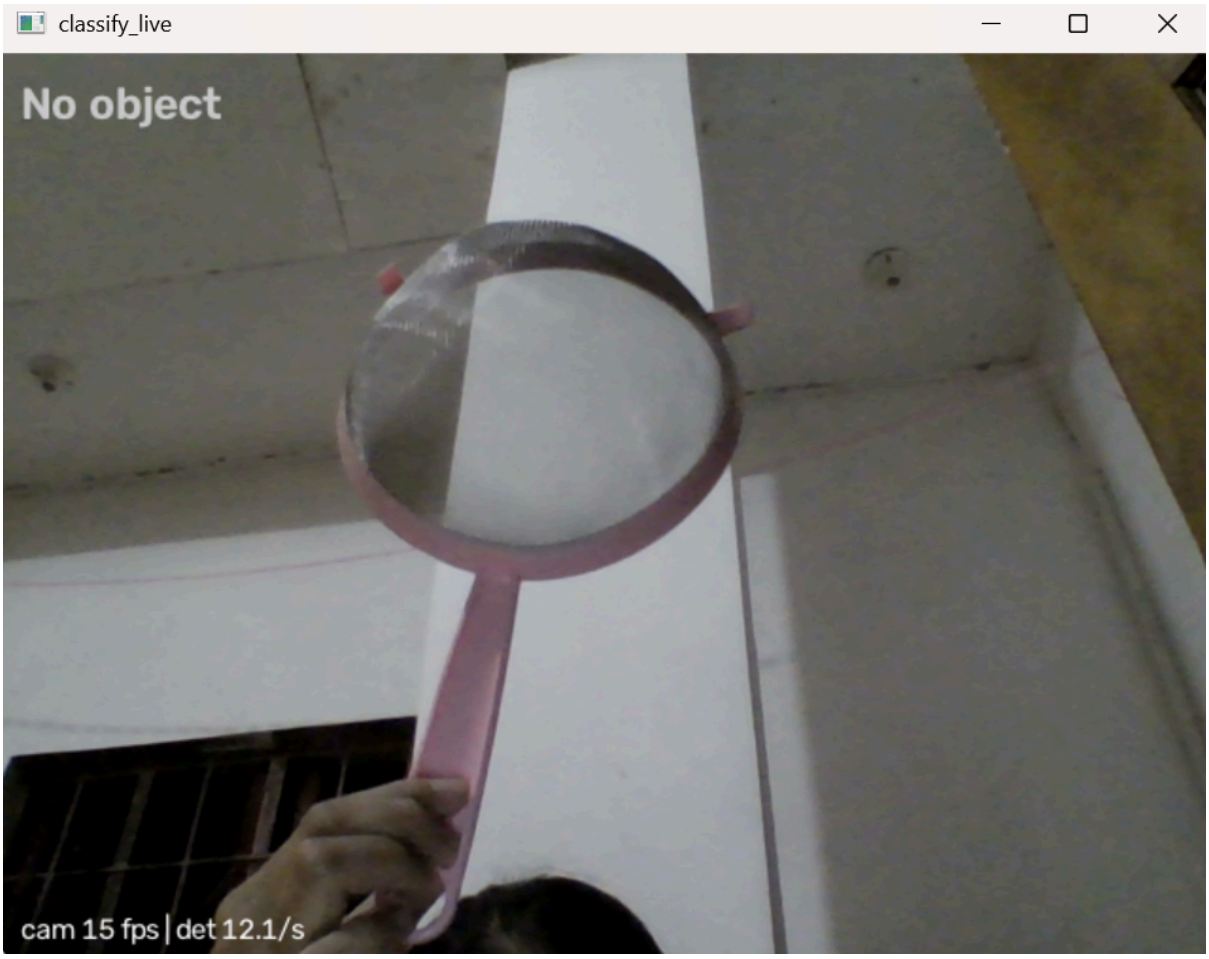


Figure 9. A round pink strainer looks similar to a ball but is not one. The system shows **no object**, so the round shape alone did not cause a false “ball” label.

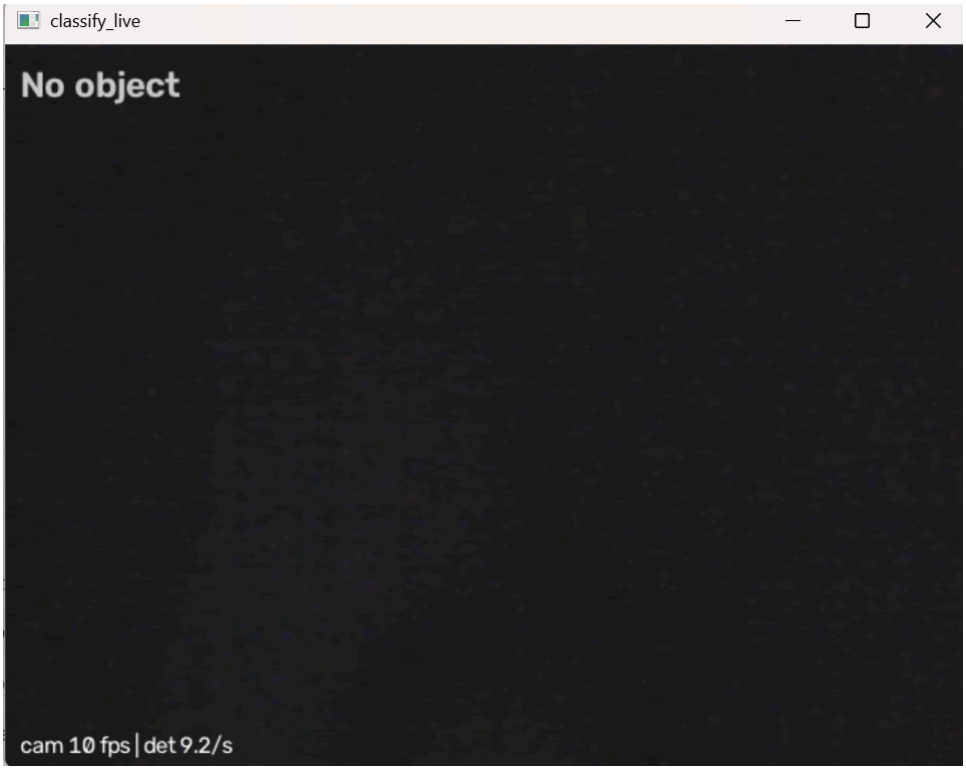


Figure 10. In a very dark room the frame is almost black (cam 10 fps). The object could not be seen, so the system showed **No object**. This is a failure: the system needs adequate lighting.

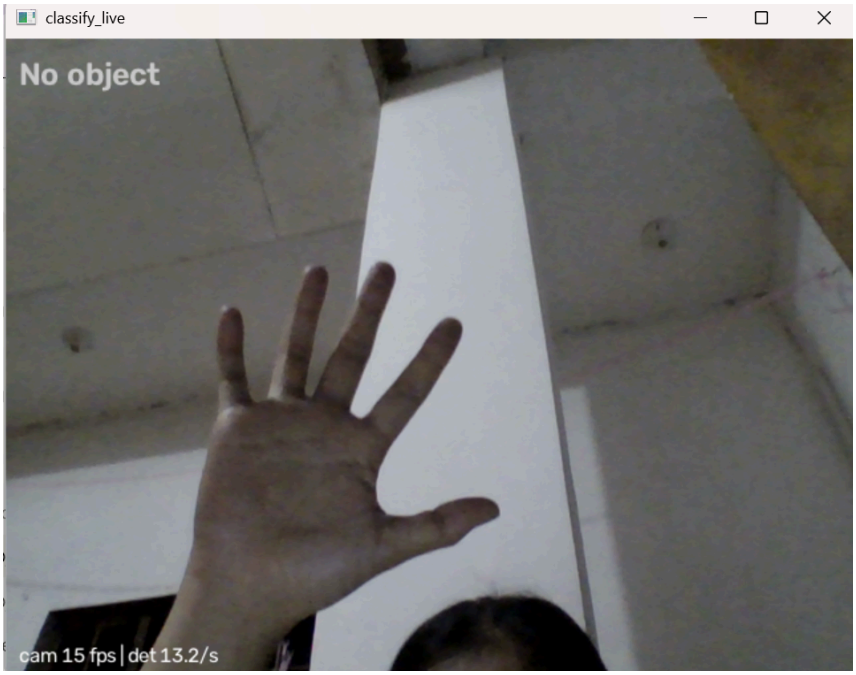


Figure 11. An open hand fills much of the frame with no object held. The system shows **No object**, so the hand is not mistaken for a bottle, can, box or ball — even though the model has no other class.

Compared with Section 6.3, recognition did not degrade for unseen objects — none were wrongly labelled. Performance drops in poor lighting, where the image is too dark for detection; the camera frame rate also fell to 10 fps because the webcam lengthens its exposure in the dark.

6.6 Final Project Test 1 — Figure-8 Line Detection and Tracking

Final Project only.

Document the group's performance during the required **Figure-8 Line Detection and Tracking Test**.

Include:

- Testing procedure
- Computer Vision output
- Robot behavior
- Test results
- Problems encountered
- Supporting screenshots/photographs

6.7 Final Project Test 2 — Group Project Specialization

Final Project only.

Document the group's performance during its assigned specialization test. Refer to Group Project Specializations for the specific testing requirements.

Include:

- Testing procedure
- Computer Vision output
- Robot decisions/actions
- Test results
- Problems encountered
- Supporting screenshots/photographs

7. RESULTS AND DISCUSSION

7.1 Computer Vision Results

The system meets the Midterm requirement of recognizing **4 object classes**. Offline, the classifier is correct on **95.5%** of the 201 validation images, with every class above 89% precision and 90% recall. Live, all four classes were recognized and CONFIRMED at **97–99% confidence**, while an empty scene and two unseen objects (a toothbrush and a round strainer)

correctly produced “No object.” The one live failure was a very dark room, where nothing could be detected. Overall, the model is reliable enough to give the Final robot the class label it needs for sorting, provided the sorting area is well lit.

7.2 Analysis of Results

What worked well:

- **Can** — 100% recall: all 30 validation cans were found, even though it is the smallest class. Class weighting (1.63) worked as intended.
- **Box** — 100% precision: nothing else was ever predicted as a box.
- **Live recognition** — every object was CONFIRMED at 97–99%, and the hand holding the object stayed outside the bounding box.
- **No false detections in an empty scene** — the system showed “No object” and ran at 12.9 detection checks per second.

What did not work as well, and why:

- **Bottles predicted as balls** — the largest error (5 of 60 bottles). Clear plastic and crushed bottles have rounded, shiny surfaces that resemble balls, and transparency lets the background show through.
- **Ball acts as the “fallback” class** — 7 of the 9 errors were predicted as ball. With 9 very different ball types, the model learned a broad idea of “ball,” so unclear images tend to land there.
- **Domain gap** — most training images come from public datasets, so lighting and backgrounds differ from the group's webcam.
- **Speed** — detection ran at 4.2–6.7 checks per second with an object in view on the laptop CPU, so a label takes about 1 second to be CONFIRMED.

7.3 Errors and Failure Cases

Failure	Count (validation)	Likely cause	Effect on sorting
Bottle predicted as ball	5 of 60 (validation)	Rounded, shiny or crushed bottles look ball-like; transparent plastic shows the background	Bottle would go to the ball bin

Box predicted as ball	2 of 52 (validation)	Unusual angle or rotated augmented images where the box shape is unclear	Box would go to the ball bin
Bottle predicted as can	1 of 60 (validation)	Short cylindrical bottle similar in shape to a can	Bottle would go to the can bin
Ball predicted as bottle	1 of 59 (validation)	Uncommon ball type or partial view	Ball would go to the bottle bin
Object not detected in darkness	Live test U3	Frame too dark for YOLO-World to find the object	Object would not be sorted until lighting improves

A hand alone was not detected as an object in live tests (Figures 8, 9 and 11), even though the model has no other class.

In total, 9 of 201 validation images (4.5%) were misclassified. In live use, the 0.80 confidence threshold and the 5-check confirmation reduce the effect of single wrong predictions, because a wrong label must be repeated with high confidence before it is shown as CONFIRMED.

7.4 Midterm-to-Final Improvements

Final Project only.

Clearly identify the improvements made after the Midterm.

A comparison table is recommended:

Component	Midterm	Final	Improvement
Dataset			
CV Method			
Model			
Performance			
Robot Integration	Not yet required		

Decision-Making	Not yet required		

7.5 Overall System Performance

Final Project only.

Discuss the performance of the complete **SEE → DECIDE → ACT** system.

Explain whether the Computer Vision component successfully provided the visual information necessary for autonomous robot behavior.

8. PROBLEMS ENCOUNTERED AND LIMITATIONS

Problems encountered during development:

- **Risk of arms and hands detected as objects.** With only four classes, the classifier must call every crop a bottle, can, box or ball. The 0.80 confidence threshold and keeping hands out of the frame reduce this; collect_data.py already supports an other class (key 5) for the fix. In testing, the 0.80 threshold prevented this (Figure 11).
- **Flickering boxes and labels.** Single-frame results jumped between classes and disappeared on brief misses. This was solved with the tracker: box smoothing, a 0.8 s hold, a lower 0.50 threshold for an object already tracked, and a 7-check majority vote.
- **Low frame rate on a CPU.** Running detection on every frame made the video lag. Moving detection to a separate thread, opening the camera in MJPG at 640×480, using detector size 416, and classifying all crops in one batch kept the video smooth.
- **Messy public data.** Images of mixed sizes, formats and some corrupt files. The training script letterboxes instead of stretching, skips unsupported formats and drops corrupt files automatically.
- **Unbalanced classes.** Can has 154 images against 300 for bottle; class weights (1.63 for can) compensate during training.

Current limitations:

- No separate test set; offline results come from the validation set.
- No other / background class in the trained model.
- One object reported per frame.
- Mostly public-dataset images rather than images from the group's own camera.
- YOLO-World is used without retraining and needs the ultralytics package and its 26 MB weights file.
- CPU-only inference gives roughly 4–13 detection checks per second.
- Needs adequate lighting; in a very dark room the object is not detected (Figure 10).
- The robot and sorting mechanism are not yet built.

9. CONCLUSION AND RECOMMENDATIONS

9.1 Conclusion

The group built a working vision system for the Visual Object Sorting Robot that **sees and recognizes four object classes — bottle, can, box and ball — in real time** from a webcam or a still image. A 1,011-image dataset was organized into class folders and split 80/20 with stratification. A MobileNetV2 model trained with two-phase transfer learning, class weighting and heavy augmentation reached **95.5% validation accuracy** and a **96.0% average F1-score**, with can (100% recall) and box (100% precision) the strongest classes. Combined with a YOLO-World detector and a tracker, it recognized and CONFIRMED all four classes live at 97–99% confidence and reported “No object” for an empty scene. This satisfies the Midterm goal of the specialization and gives the Final robot the class label and object position it needs. Unseen objects were correctly ignored. The main remaining weaknesses are bottles being mistaken for balls (5 of 60) and failure in very dark lighting.

9.2 Recommendations

1. **Add an other class** — 30+ photos each of hands, arms, the empty table and clutter, collected with collect_data.py (key 5), then retrain.
2. **Create a true test set** — hold out 10–15% of each class (e.g. 70/15/15) that is never used for training or early stopping.
3. **Collect images from the actual robot camera** — at least 50 per class in the final sorting area and lighting.
4. **Reduce bottle–ball confusion** — add more clear and crushed bottle photos, the class most often mistaken for ball.
5. **Add a light source** (e.g. an LED) above the sorting area so recognition does not depend on room lighting.
6. **Prepare for the Final** — map each class to a bin (e.g. bottle → A, can → B, box → C, ball → D), send the CONFIRMED label over serial to the microcontroller, and use the box centre to position the robot.
7. **Speed up if needed** — --imgsz 320 and --no-refine, or a GPU.

REFERENCES

Abadi, M., Barham, P., Chen, J., Chen, Z., Davis, A., Dean, J., … Zheng, X. (2016). TensorFlow: A system for large-scale machine learning. In *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation* (pp. 265–283). USENIX Association.

Bradski, G. (2000). The OpenCV library. *Dr. Dobb's Journal of Software Tools, 25*(11), 120–125.

Cheng, T., Song, L., Ge, Y., Liu, W., Wang, X., & Shan, Y. (2024). YOLO-World: Real-time open-vocabulary object detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (pp. 16901–16911).

Chollet, F., et al. (2015). *Keras* [Computer software]. https://keras.io

Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., & Fei-Fei, L. (2009). ImageNet: A large-scale hierarchical image database. In *2009 IEEE Conference on Computer Vision and Pattern Recognition* (pp. 248–255). IEEE.

Jocher, G., Qiu, J., & Chaurasia, A. (2023). *Ultralytics YOLO* (Version 8) [Computer software]. https://github.com/ultralytics/ultralytics

Rother, C., Kolmogorov, V., & Blake, A. (2004). “GrabCut”: Interactive foreground extraction using iterated graph cuts. *ACM Transactions on Graphics, 23*(3), 309–314.

Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.-C. (2018). MobileNetV2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (pp. 4510–4520).

APPENDICES

Appendix A — How to Run the System

- pip install -r requirements.txt # Python 3.9–3.12
  python collect_data.py # 1 bottle, 2 can, 3 box, 4 ball, 5 other, q quit
  python train.py # trains on ./dataset, writes model.keras + labels.txt
  python train.py --data my_folder --epochs 15
  python classify_live.py # live webcam recognition
  python classify_live.py photo.jpg # saves photo_labeled.jpg

- **Appendix B — Command-Line Options of `classify_live.py`**

1. Option	2. Default	3. Purpose
`--mode`	yolo	Object finder: yolo, auto, background or none
`--camera`	0	Camera index
`--conf`	0.80	Confidence to start tracking a new object
`--keep-conf`	0.50	Confidence to keep tracking an object
`--hold`	0.8	Seconds the box stays after the object is last seen
`--det-conf`	0.10	Minimum YOLO-World confidence for candidates
`--imgsz`	416	YOLO-World input size
`--no-refine`	off	Skip the GrabCut outline for more speed

- **Appendix C — Project Files**

File	Size	Content
`model.keras`	21.9 MB	Trained MobileNetV2 classifier (4 classes)
`labels.txt`	19 B	Class order: ball, bottle, box, can
`yolov8s-worldv2.pt`	25.9 MB	YOLO-World detector weights

dataset/dataset.rar	35.1 MB	1,011 training images in 4 class folders

- **Appendix D — Additional Screenshots and Test Records**

  Console output of python evaluate.py:

```
Accuracy: 95.5%  (201 images)

Class      Precision    Recall        F1  Images
ball           89.2%     98.3%     93.5%      59
bottle         98.2%     90.0%     93.9%      60
box           100.0%     96.2%     98.0%      52
can            96.8%    100.0%     98.4%      30
Average        96.0%     96.1%     96.0%     201

Confusion matrix (rows = true, columns = predicted):
               ball   bottle      box      can
ball             58        1        0        0
bottle            5       54        0        1
box               2        0       50        0
can               0        0        0       30
```